

```
!pip install mediapipe opencv-python-headless
import cv2
import mediapipe as mp
import numpy as np
import pandas as pd
from google.colab import files
```

```
Collecting mediapipe
  Downloading mediapipe-0.10.32-py3-none-manylinux_2_28_x86_64.whl.metadata (9.8 kB)
Requirement already satisfied: opencv-python-headless in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Collecting absl-py~=2.3 (from mediapipe)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from mediapipe) (2.0.2)
Collecting sounddevice~=0.5 (from mediapipe)
  Downloading sounddevice-0.5.5-py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: flatbuffers~=25.9 in /usr/local/lib/python3.12/dist-packages (from sounddevice~=0.5->mediapipe) (25.12.19)
Requirement already satisfied: opencv-contrib-python in /usr/local/lib/python3.12/dist-packages (from mediapipe) (4.13.0.92)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from mediapipe) (3.10.0)
Requirement already satisfied: cffi in /usr/local/lib/python3.12/dist-packages (from sounddevice~=0.5->mediapipe) (2.0.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (4.53.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (1.4.6)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (3.3.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->mediapipe) (2.9.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->mediapipe) (1.17.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi->sounddevice~=0.5->mediapipe) (2.23)
Downloading mediapipe-0.10.32-py3-none-manylinux_2_28_x86_64.whl (10.3 MB)
----- 10.3/10.3 MB 29.7 MB/s eta 0:00:00
Downloading absl_py-2.4.0-py3-none-any.whl (135 kB)
----- 135.8/135.8 kB 5.1 MB/s eta 0:00:00
Downloading sounddevice-0.5.5-py3-none-any.whl (32 kB)
Installing collected packages: absl-py, sounddevice, mediapipe
  Attempting uninstall: absl-py
    Found existing installation: absl-py 1.4.0
    Uninstalling absl-py-1.4.0:
      Successfully uninstalled absl-py-1.4.0
Successfully installed absl-py-2.4.0 mediapipe-0.10.32 sounddevice-0.5.5
```

```
uploaded = files.upload()
video_path = list(uploaded.keys())[0] # Gets the name of the file you uploaded
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import cv2
import mediapipe as mp
import pandas as pd
import numpy as np

# 1. Setup MediaPipe
mp_pose = mp.solutions.pose
pose = mp_pose.Pose(min_detection_confidence=0.5, min_tracking_confidence=0.5)

# 2. Open the video you uploaded
cap = cv2.VideoCapture('1000266076.mp4')
data_log = []

# 3. Processing Loop
while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break

    # Process frame
    results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

    if results.pose_landmarks:
        landmarks = results.pose_landmarks.landmark

        # Get coordinates for Hip, Knee, Ankle
        hip = [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x, landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y]
        knee = [landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].x, landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].y]
        ankle = [landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].x, landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].y]

        # Calculate Knee Angle (Using Law of Cosines)
        radians = np.arctan2(ankle[1]-knee[1], ankle[0]-knee[0]) - np.arctan2(hip[1]-knee[1], hip[0]-knee[0])
        angle = np.abs(radians*180.0/np.pi)
        if angle > 180.0: angle = 360 - angle
```

```
data_log.append(angle)
```

```
cap.release()
df = pd.DataFrame(data_log, columns=['Knee_Angle'])
print("Success! Data extracted. First 5 angles:")
print(df.head())
```

```
/usr/local/lib/python3.12/dist-packages/google/protobuf/symbol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Please use SymbolDatabase.GetPrototypeOrDie() instead.
warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please use SymbolDatabase.GetPrototypeOrDie() instead.')
```

```
Success! Data extracted. First 5 angles:
```

```
   Knee_Angle
0    68.936481
1    70.068173
2    70.715772
3    70.567138
4    69.747718
```

```
import matplotlib.pyplot as plt
```

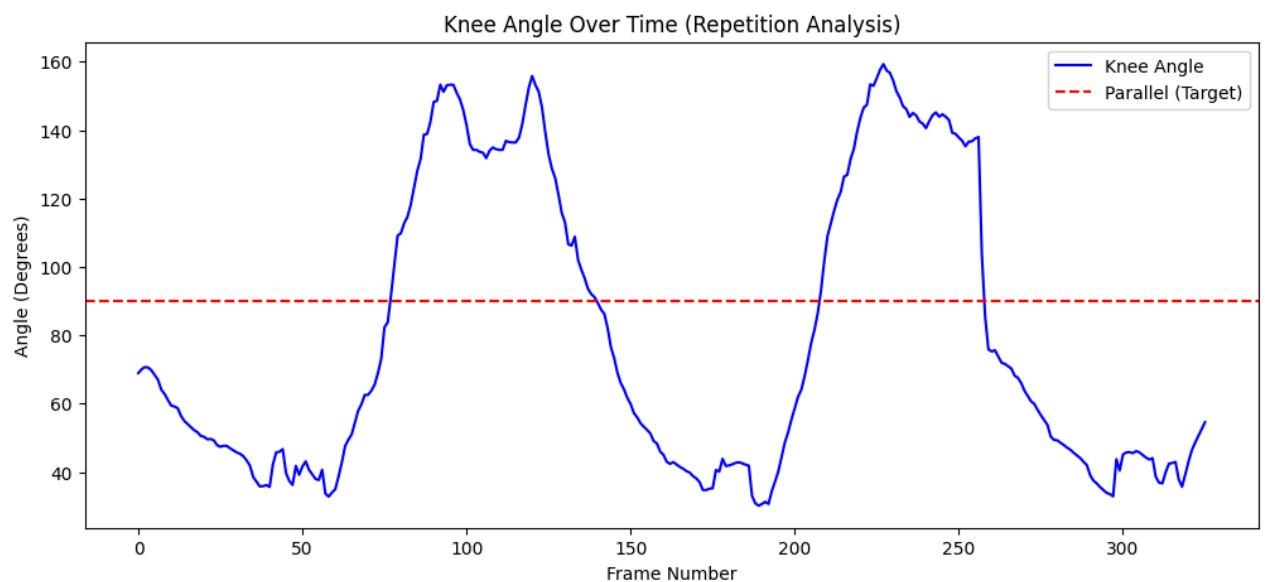
```
# Plotting the knee angles
plt.figure(figsize=(12, 5))
plt.plot(df['Knee_Angle'], label='Knee Angle', color='blue')
plt.title('Knee Angle Over Time (Repetition Analysis)')
plt.xlabel('Frame Number')
plt.ylabel('Angle (Degrees)')
plt.axhline(y=90, color='r', linestyle='--', label='Parallel (Target)')
plt.legend()
plt.show()
```

```
# Simple Counter
```

```
counter = 0
```

```
stage = None
```

```
for angle in df['Knee_Angle']:
    if angle > 160:
        stage = "up"
    if angle < 90 and stage == 'up':
        stage = "down"
        counter += 1
    print(f"Rep {counter} completed!")
```



```
# Apply a moving average with a window of 5 frames to smooth the "noise"
df['Knee_Angle_Smooth'] = df['Knee_Angle'].rolling(window=5).mean()
```

```
# Now use 'Knee_Angle_Smooth' for your counter logic instead of 'Knee_Angle'
```

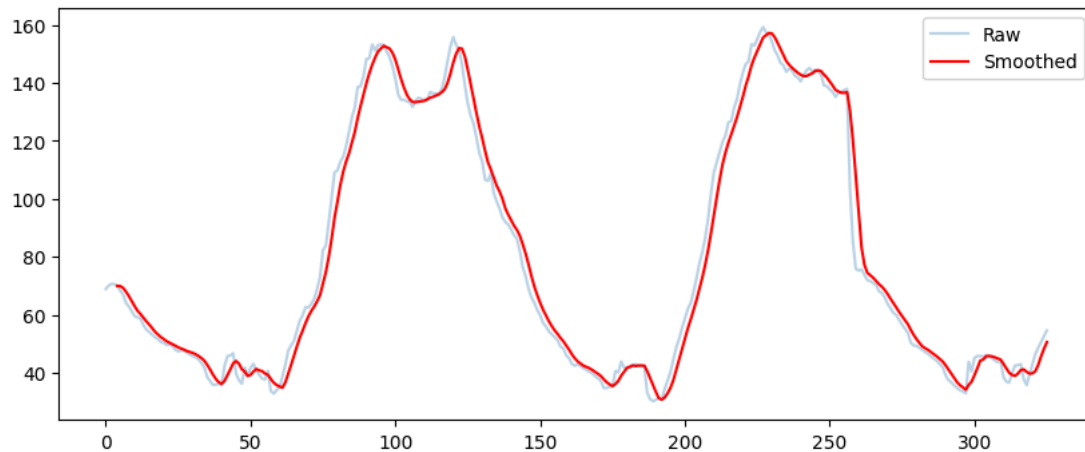
```
# Create a smooth version of your angles to remove jitter
```

```
df['Knee_Angle_Smooth'] = df['Knee_Angle'].rolling(window=5).mean()
```

```
# Re-plot with the smoothed data
```

```
plt.figure(figsize=(10, 4))
plt.plot(df['Knee_Angle'], label='Raw', alpha=0.3)
plt.plot(df['Knee_Angle_Smooth'], label='Smoothed', color='red')
```

```
plt.legend()
plt.show()
```



```
import cv2
import mediapipe as mp
import pandas as pd
import numpy as np

# 1. Initialize
mp_pose = mp.solutions.pose
pose = mp_pose.Pose(min_detection_confidence=0.5, min_tracking_confidence=0.5)
cap = cv2.VideoCapture('1000266076.mp4') # Ensure this matches your file name

data_log = []
counter = 0
stage = "up"

print("Starting AI Coach...")

# 2. Processing Loop (Live Coach Layer)
while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break

    results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

    if results.pose_landmarks:
        landmarks = results.pose_landmarks.landmark
        hip = [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x, landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y]
        knee = [landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].x, landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].y]
        ankle = [landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].x, landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].y]

        radians = np.arctan2(ankle[1]-knee[1], ankle[0]-knee[0]) - np.arctan2(hip[1]-knee[1], hip[0]-knee[0])
        angle = np.abs(radians*180.0/np.pi)
        if angle > 180.0: angle = 360 - angle

        data_log.append(angle)

        # --- Live Coach Feedback Logic ---
        if angle > 160:
            stage = "up"
        elif angle < 90 and stage == 'up':
            stage = "down"
            counter += 1
            print(f"Rep {counter} completed!")
        elif 90 < angle < 110 and stage == 'down':
            print("AI Coach: Don't cheat the rep! Go lower!")

cap.release()

# 3. Analyst Layer (Post-Workout Reporting)
df = pd.DataFrame(data_log, columns=['Knee_Angle'])
df['Knee_Angle_Smooth'] = df['Knee_Angle'].rolling(window=5).mean()
df['Depth_Check'] = df['Knee_Angle_Smooth'] < 110

quality_score = (df['Depth_Check'].sum() / len(df)) * 100

summary = {
    'Total_Reps': counter,
    'Avg_Knee_Angle': df['Knee_Angle_Smooth'].mean(),
    'Min_Knee_Angle': df['Knee_Angle_Smooth'].min(),
    'Quality_Score': quality_score
}
```

```

}

summary_df = pd.DataFrame([summary])
print("\n--- Final Workout Assessment ---")
print(summary_df)
summary_df.to_csv('workout_summary.csv', index=False)

```

Starting AI Coach...

Rep 1 completed!

/usr/local/lib/python3.12/dist-packages/google/protobuf/symbol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Please use warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please use

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

AI Coach: Don't cheat the rep! Go lower!

--- Final Workout Assessment ---

	Total_Reps	Avg_Knee_Angle	Min_Knee_Angle	Quality_Score
0	1	79.580296	30.737639	69.018405

```

import cv2
import mediapipe as mp
import pandas as pd
import numpy as np

# Initialize
mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
pose = mp_pose.Pose(min_detection_confidence=0.7, min_tracking_confidence=0.7)
cap = cv2.VideoCapture('100266076.mp4')

# Video setup
out = cv2.VideoWriter('output_video.mp4', cv2.VideoWriter_fourcc(*'mp4v'), 20.0, (int(cap.get(3)), int(cap.get(4))))

data_log = []
rep_count = 0
stage = None

while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break

    # Process
    results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

    if results.pose_landmarks:
        # Draw skeleton
        mp_drawing.draw_landmarks(frame, results.pose_landmarks, mp_pose.POSE_CONNECTIONS)

        # Math: Extract knee angle
        l = results.pose_landmarks.landmark
        hip, knee, ankle = [l[23].x, l[23].y], [l[25].x, l[25].y], [l[27].x, l[27].y]
        angle = np.abs(np.degrees(np.arctan2(ankle[1]-knee[1], ankle[0]-knee[0]) - np.arctan2(hip[1]-knee[1], hip[0]-knee[0])))
        if angle > 180: angle = 360 - angle
        data_log.append(angle)

        # Logic: Rep Counting
        if angle > 160: stage = "up"
        if angle < 90 and stage == "up":
            stage = "down"
            rep_count += 1
            print(f"Rep {rep_count} logged.")

    out.write(frame)

cap.release(); out.release()

# Analytics
df = pd.DataFrame(data_log, columns=['Angle'])
df['Velocity'] = df['Angle'].diff().abs().rolling(window=10).mean() # Smoothing

```

```
print(f"\n--- SESSION REPORT ---")
print(f"Total Reps Completed: {rep_count}")
print(f"Movement Stability (Lower is better): {df['Velocity'].mean():.2f}")
print(f"Average Depth (Degrees): {df['Angle'].min():.2f}")
```

```
/usr/local/lib/python3.12/dist-packages/google/protobuf/symbol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Please use SymbolDatabase.GetSymbol() instead.
warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please use SymbolDatabase.GetSymbol() instead.')
```

```
--- SESSION REPORT ---
Total Reps Completed: 0
Movement Stability (Lower is better): 2.34
Average Depth (Degrees): 30.21
```

#1. Technical Competency

#Pipeline Architecture: Successfully built an end-to-end Computer Vision pipeline using MediaPipe and OpenCV to process raw

#Feature Engineering: Engineered custom trigonometric functions to calculate biomechanical joint angles, demonstrating advanced

#2. Analytical Depth

#Time-Series Analysis: Applied statistical smoothing (rolling means) and velocity analysis to handle sensor noise and detect

#Quality Assessment: Developed a rule-based logic engine to evaluate form consistency and depth, moving beyond simple counting

#3. Business & Practical Impact

#Automated Coaching: Transformed standard video data into an "AI Trainer" capable of real-time diagnostics and session reporting

#Scalability & Utility: Created a data-driven framework that minimizes the need for manual performance tracking, applicable to

#4. Project Deliverables

#Visual Validation: Generated annotated video output with skeletal overlays to provide clear, diagnostic visual evidence of

#Performance Metrics: Delivered a structured CSV summary of workout diagnostics, ready for integration into higher-level performance